

- ArrayList
- LinkedList
- Stack
- Queue
- PriorityQueue
- Map
- Set

Java Collection Framework (JCF)

[ArrayList দিহি
list of objects create করতে হবে
! example
! common methods

Object creation
in JCF

class use করলে index দিহি accessible
হবে।

SET

'Set' is interface

'HashSet' is class

2 ways
↙ ↘
use ref use class

- ↑ use this as reference
- ↑ no particular order (unordered)
- ↑ only unique values
- ↑ cannot be accessed via index number

interface ref
↓

```
Set <String> s = new HashSet();
```

```
HashSet <String> s = new HashSet();
```

↑

class ref

check out from  PDF

which is a condensed version of Afg

Next lab ⇒ project display

Next Thursday ⇒ CT (Java Collection Framework)

Set > Only 'HashSet'

Shayan

```
Set <String> s = new HashSet<String>();
```

```
s.add("B");
```

```
s.add("B");
```

```
s.add("C");
```

```
s.add("A");
```

```
{ A, B, C } ← system.out.println
```

```
s.contains("D"); // Boolean true or false
```

```
s.remove("B");
```

iterating a set

foreach
loop

iterate
interface

```
h = new HashSet<String>();
```

```
for (String value : h)  
    System.out.println(value);
```

add (elem)

add All (collection)

clear()

isEmpty() ← often used

remove()

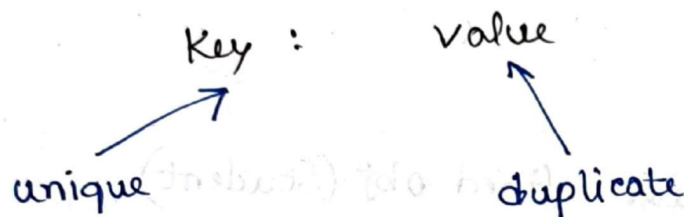
hashCode()

size()

PriorityQueue is a class but Queue is an interface

#MAP

Key-value pair $\checkmark$ data $\checkmark$



name : ABC

name : ABC

☒ No index

☒ use Keys to perform operations

interface Map <K, V>

↳ HashMap ← only this is needed

```
Map <String, Integer> m = new HashMap <> ();
```

```
m.put ("Geek1", 1);
```

```
m.put ("Geek2", 2);
```

```
System.out.println (m);
```

```
{Geek2 = 2, Geek1 = 1}
```

```
m.put (new Integer(4), "F");
```

```
m.put (4, "F");
```

```
m.remove();
```

#iterating maps

```
for (Map.Entry mapElement: m.entrySet()) {
    int key = (int) mapElement.getKey();
    ...
}
```

==

ArrayList → user defined obj (Student)

==

```
import java.util.*;

class Student {
    int id, String name; Student() {}
    Student(int id, String name) {
        this.id = id; this.name = name;
    }
    void display(ArrayList<Student> c1) {
        System.out.println(c1); for (Student s: c1) {
            System.out.println(s);
        }
    }
    public String toString() {
        return "id: " + id + ", " + "name: " + name;
    }
}
```

```
class St {
    psvm(...) {
        ArrayList<Student> c1 = new ArrayList<Student>();
        Student s1 = new Student(2011, "ABC");
        Student s2 = new Student(3011, "DEF");
    }
}
```


Shayan

```
Student s3 = new Student();
```

```
c1.add(s1);
```

```
c1.add(s2);
```

```
s3.display(c1);
```

```
}
```

```
}
```

Stack 

```
class Library {  
  String title, author;  
  int id, price;
```

class - library

↳ b-id

↳ b-title

↳ b-author

↳ b-price

class Library {

int b-id, b-price;

String b-title, b-author;

Library () {}

Library (int b-id, int b-price, String b-title, b-author) {

this.b-id = b-id;

this.b-price = b-price;

this.b-title = b-title;

this.b-author = b-author;

}

void display (Stack <Library> stk) {

for (Stack s : stk) {

System.out.println(s);

}

}

public String toString () {

return "id: " + b-id + ", title: " + b-title

+ ", author: " + b-author + ", price: " + b-price;

}

void borrow (Stack <Library> stk) {

stk.pop();

}

}

~~class~~ public class Main {

public static void main (String[] args) {

Stack <Library> s1 = new Stack <> ();

Library l1 = new Library (1, 200, "AB", "CD");